فصل ۳: تابع، ارسال پارامتر، حوزهٔ متغیر، static و ماکرو

چرا این فصل؟ تست‌های تابع تقریباً همیشه یکی از این چهار تله را دارند: «مقدار در تابع عوض شد ولی بیرون نه»، متغیر `static` که مقدارش می‌ماند، متغیر هم‌نام محلی و سراسری، و ماکروی بدون پرانتز. **اعتبار:** خروجی همهٔ کدها با `gcc` اجرا و چک شده است. **زمان مطالعه:** یک بلوک ۹۰ دقیقه‌ای.

درس ۱: تعریف و فراخوانی تابع

```
int add(int a, int b);          // اعلان (prototype)
int add(int a, int b) {         // تعریف
    return a + b;
}
```

- نوع `void` یعنی تابع مقدار برنمی‌گرداند.
- **ترتیب ارزیابی آرگومان‌ها در C مشخص نیست.** پس در `f(i++, i++)` نتیجه قطعی نیست و تست درست آن را نمی‌پرسد.
- `return` تابع را **فوراً** تمام می‌کند؛ کدهای بعد از آن اجرا نمی‌شوند.

درس ۲: ارسال پارامتر

در C همه‌چیز «با مقدار» فرستاده می‌شود

تابع یک **کپی** از آرگومان می‌گیرد. تغییر کپی روی متغیر اصلی اثری ندارد.

```
void swap_v(int a, int b) { int t = a; a = b; b = t; }      // کار نمی‌کند
void swap_p(int *a, int *b) { int t = *a; *a = *b; *b = t; } // کار می‌کند
```

با `x = 1` و `y = 2`، بعد از `swap_v(x, y)` همان `1 2` می‌ماند. بعد از `swap_p(&x, &y)` می‌شود `2 1`.

برای تغییر متغیر بیرونی، آدرسش را بفرست (اشاره‌گر). جزئیات اشاره‌گر در فصل ۵ است.

آرایه استثنای ظاهری است

وقتی آرایه را به تابع می‌دهی، **آدرس اولین عضو** فرستاده می‌شود. پس تغییر عضوها در تابع، آرایهٔ اصلی را عوض می‌کند.

```
void set0(int a[]) { a[0] = 9; }
int v[3] = {1, 2, 3};  set0(v);   // حالا v[0] == 9
```

⚠️ داخل تابع، `sizeof(a)` اندازهٔ **اشاره‌گر** است (۸ بایت در سیستم ۶۴ بیتی)، نه اندازهٔ آرایه. بیرون تابع، `sizeof(v)` برای `int v[10]` برابر ۴۰ است.

تمرین درس ۲

```
void g(int x)  { x = x * 2; }
void h(int *p) { *p = *p * 2; }
int a = 5;  g(a);  printf("%d ", a);  h(&a);  printf("%d", a);
```

> 🔑 **جواب:** پاسخ‌نامهٔ انتهای فصل، بخش ۱ (درس ۲: ارسال پارامتر)

درس ۳: حوزه (scope) و طول عمر

نوع متغیر	دیده می‌شود در	طول عمر	مقدار اولیهٔ پیش‌فرض
محلی معمولی (auto)	همان بلوک `{ }`	هر بار ورود به بلوک ساخته و در خروج نابود می‌شود	**نامعلوم (زباله)**
محلی `static`	همان بلوک	**کل اجرای برنامه**؛ مقدارش بین فراخوانی‌ها می‌ماند	۰
سراسری	کل فایل	کل اجرای برنامه	۰

دو قاعدهٔ کلیدی

۱. **مقداردهی اولیهٔ `static` فقط یک بار انجام می‌شود.** خط `static int c = 0;` در فراخوانی دوم دوباره c را صفر نمی‌کند.

۲. **پوشاندن (shadowing):** متغیر محلی هم‌نام، متغیر سراسری یا بیرونی را در آن بلوک **پنهان** می‌کند. ولی تابعی که بیرون تعریف شده، همیشه سراسری را می‌بیند.

مثال:

```
int x = 5;                                  // سراسری
void hx(void) { printf("%d ", x); }         // همیشه سراسری را می‌بیند
int main(void) {
    int x = 10;
    hx();                                   // 5
    { int x = 20; printf("%d ", x); }       // 20
    printf("%d", x);                        // 10
}
```

خروجی `5 20 10`.

تمرین درس ۳

```
// 1
int counter(void) { static int c = 0; c++; return c; }
counter();  counter();  printf("%d", counter());
// 2
int fa(void) { int a = 0; static int s = 0; a++; s++; return a*10 + s; }
fa();  fa();  printf("%d", fa());
```

> 🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۲ (درس ۳: حوزه (scope) و طول عمر)

درس ۴: ماکرو (`#define`)

ماکرو **جایگزینی متنی** است و قبل از کامپایل انجام می‌شود؛ تابع نیست و اولویت عملگرها را حفظ نمی‌کند.

ماکرو	`SQ(1+2)`	`100 / SQ(5)`
`#define SQ(x) x*x`	`1+2*1+2 = 5`	`100/5*5 = 100`
`#define SQ(x) ((x)*(x))`	`9`	`4`

قاعده: هر پارامتر را در پرانتز بگذار و کل عبارت را هم در پرانتز. ⚠️ حتی ماکروی درست هم با `++` خطرناک است، چون آرگومان ممکن است **دو بار** ارزیابی شود (سؤال ۱۰ آزمون).

📝 آزمون فصل ۳

این درس در کنکور ارشد کامپیوتر سؤال سابقه ندارد. پس این ۱۰ تست **تألیفی** هستند و خروجی همه با `gcc` تأیید شده است.

زمان: ۱۵ دقیقه. خروجی هر کد چیست؟

۱.

```
int f(int n) { static int s = 1; s *= n; return s; }
// main:
f(2); f(3); printf("%d", f(4));
```

۱) `4` ۲) `8` ۳) `24` ۴) `12`

۲.

```
void g(int x) { x = x * 2; }
// main:
int a = 5; g(a); g(a); printf("%d", a);
```

۱) `5` ۲) `10` ۳) `20` ۴) مقدار نامعلوم

۳.

```
void g(int *p) { *p = *p * 2; }
// main:
int a = 5; g(&a); g(&a); printf("%d", a);
```

۱) `5` ۲) `10` ۳) `20` ۴) آدرس a

۴.

```
#define DOUBLE(x) x + x
printf("%d", 3 * DOUBLE(4));
```

۱) `24` ۲) `16` ۳) `14` ۴) `12`

۵.

```
int x = 1;
int f(void) { return x * 10; }
int main(void) { int x = 2; printf("%d", f() + x); }
```

۱) `12` ۲) `22` ۳) `11` ۴) `21`

۶.

```
void z(int a[], int n) { for (int i = 0; i < n; i++) a[i] = a[i] * i; }
// main:
int v[4] = {5, 5, 5, 5}; z(v, 4); printf("%d %d", v[0], v[3]);
```

۱) `5 5` ۲) `0 15` ۳) `5 15` ۴) `0 20`

۷.

```
int t(void) { static int k = 10; return k--; }
// main:
int s = 0; for (int i = 0; i < 3; i++) s += t(); printf("%d", s);
```

۱) `30` ۲) `27` ۳) `24` ۴) `9`

۸.

```
int g = 0;
int h(int v) { g += v; return g; }
// main:
int a = h(3); int b = h(4); printf("%d %d %d", a, b, g);
```

۱) `3 4 7` ۲) `3 7 7` ۳) `3 7 0` ۴) `7 7 7`

۹.

```
for (int i = 0; i < 3; i++) {
    int a = 0; static int b = 0;
    a += i; b += i;
    if (i == 2) printf("%d %d", a, b);
}
```

۱) `2 2` ۲) `3 3` ۳) `2 3` ۴) `3 2`

۱۰.

```
#define MAX(a,b) ((a) > (b) ? (a) : (b))
int x = 5, y = 3;
int m = MAX(x++, y);
printf("%d %d", m, x);
```

۱) `5 6` ۲) `6 7` ۳) `5 7` ۴) `6 6`

> 🔑 **جواب‌ها و راه‌حل:** پاسخ‌نامهٔ انتهای فصل، بخش ۳ (آزمون فصل ۳)

🗂 کارت خلاصهٔ فصل ۳

- C همه‌چیز را با مقدار می‌فرستد. برای تغییر متغیر بیرونی، آدرسش را بفرست.

- آرایه به‌صورت آدرس فرستاده می‌شود، پس تغییر عضوها بیرون دیده می‌شود. `sizeof` آرایه داخل تابع اندازهٔ اشاره‌گر است.
- محلی معمولی هر بار از نو ساخته می‌شود. `static` یک بار مقداردهی می‌شود و مقدارش می‌ماند. سراسری و static پیش‌فرض صفرند.
- متغیر محلی هم‌نام، سراسری را می‌پوشاند؛ ولی تابع‌های دیگر سراسری را می‌بینند.
- ماکرو جایگزینی متنی است. بدون پرانتز اولویت به هم می‌ریزد و با `++` آرگومان ممکن است دو بار اجرا شود.

🔑 پاسخ‌نامهٔ فصل

بخش ۱: درس ۲: ارسال پارامتر

`10 5` . تابع g روی کپی کار می‌کند و تابع h از طریق آدرس، خود a را دو برابر می‌کند.

بخش ۲: درس ۳: حوزه (scope) و طول عمر

۱. `3`

۲. `13` . متغیر a هر بار از صفر شروع می‌شود و ۱ می‌شود؛ s در فراخوانی سوم ۳ است.

بخش ۳: آزمون فصل ۳

سؤال	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰
جواب	۳	۱	۳	۲	۱	۲	۲	۲	۳	۲

۱. s مقدارش را نگه می‌دارد: `1·2·3·4 = 24` .

۲. g روی کپی کار می‌کند، پس a همان **۵** است.

۳. از طریق اشاره‌گر دو بار دو برابر می‌شود: `5 → 10 → 20` .

۴. جایگزینی متنی: `3 * 4 + 4 = 16` .

۵. f متغیر سراسری x (یعنی ۱) را می‌بیند: `10 + 2 = 12` .

۶. `a[i] = 5·i` ، پس `v[0] = 0` و `v[3] = 15` .

۷. `k--` اول مقدار قدیمی را برمی‌گرداند: `10 + 9 + 8 = 27` .

۸. `a = 3` ، بعد `g = 7` پس `b = 7` ، و در آخر `g = 7` .

۹. a هر دور از صفر شروع می‌شود و در دور آخر ۲ است. b جمع می‌شود: `0 + 1 + 2 = 3` .

۱۰. ماکرو باز می‌شود به `((x++) > (y) ? (x++) : (y))` . اول `x++` مقدار ۵ را می‌دهد (و x می‌شود ۶). چون `5 > 3` ، دوباره `x++` ارزیابی می‌شود که ۶ را برمی‌گرداند و x را ۷ می‌کند. نتیجه `m = 6` و `x = 7` . (بعد از عملوند اول `?:` نقطهٔ ترتیب هست، پس رفتار تعریف‌شده است.)